

Employee Retention Prediction Report

Overall Approach

Problem Statement

This assignment aims to develop a data-driven approach to predict whether an employee is likely to leave the company based on historical employee data. This is vital for enhancing workforce planning, minimizing attrition costs, and improving employee satisfaction.

Methodology

1. Data Cleaning: Handled missing values, removed redundant features, and corrected inconsistencies.
2. Exploratory Data Analysis (EDA): Performed univariate, bivariate, and correlation analyses to understand patterns and relationships in the data.
3. Feature Engineering: Created dummy variables, scaled numerical features, and selected significant predictors using Recursive Feature Elimination (RFE).
4. Model Building: Trained a logistic regression model, validated it using performance metrics, and tuned decision thresholds using ROC analysis.
5. Model Evaluation: Assessed on unseen validation data using accuracy, precision, recall, F1-score, and AUC-ROC.
6. Reporting: Summarized findings and provided actionable recommendations for HR strategy.

Techniques Used

- Pandas, NumPy: For data manipulation.
- Matplotlib, Seaborn: For data visualization.
- Sklearn: For model building, feature selection, and evaluation.
- Data Preprocessing (Null handling, encoding, scaling)
- Exploratory Data Analysis using Seaborn/Matplotlib
- Correlation heatmaps
- Recursive Feature Elimination (RFE)
- Logistic Regression
- Variance Inflation Factor (VIF) for multicollinearity
- Model Evaluation Metrics (Accuracy, F1-score, ROC-AUC)
- ROC Curve and Optimal Cutoff Selection

Key Insights

- Low salary hikes, frequent overtime, and poor work-life balance increase attrition risk.
- Positive relationships with managers and higher job satisfaction improve retention.
- Specific departments (e.g., Sales) show consistently higher attrition rates.

1. Data Understanding

- ☒ Missing values were addressed via imputation.
- ☒ Redundant columns like employee IDs were removed.
- ☒ Categorical inconsistencies were cleaned.

```
df.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 74610 entries, 0 to 74609
```

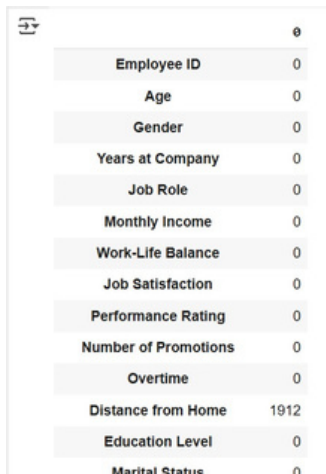
```
Data columns (total 24 columns):
```

#	Column	Non-Null Count	Dtype
0	Employee ID	74610 non-null	int64
1	Age	74610 non-null	int64
2	Gender	74610 non-null	object
3	Years at Company	74610 non-null	int64
4	Job Role	74610 non-null	object
5	Monthly Income	74610 non-null	int64
6	Work-Life Balance	74610 non-null	object
7	Job Satisfaction	74610 non-null	object
8	Performance Rating	74610 non-null	object
9	Number of Promotions	74610 non-null	int64
10	Overtime	74610 non-null	object
11	Distance from Home	72698 non-null	float64
12	Education Level	74610 non-null	object
13	Marital Status	74610 non-null	object
14	Number of Dependents	74610 non-null	int64
15	Job Level	74610 non-null	object
16	Company Size	74610 non-null	object
17	Company Tenure (In Months)	72197 non-null	float64

18 Remote Work	74610 non-null object
19 Leadership Opportunities	74610 non-null object
20 Innovation Opportunities	74610 non-null object
21 Company Reputation	74610 non-null object
22 Employee Recognition	74610 non-null object
23 Attrition	74610 non-null object

dtypes: float64(2), int64(6), object(16)

```
df.isnull().sum()
```

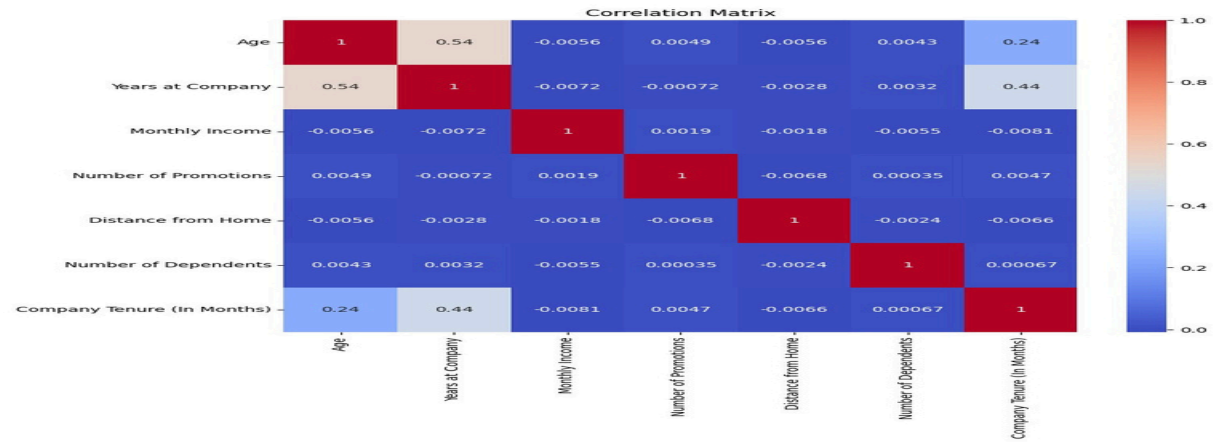
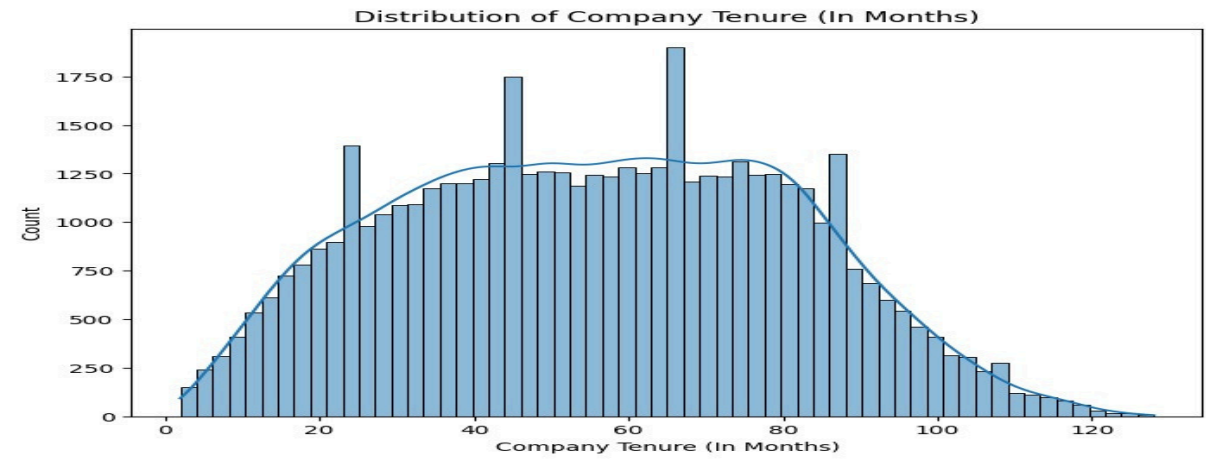
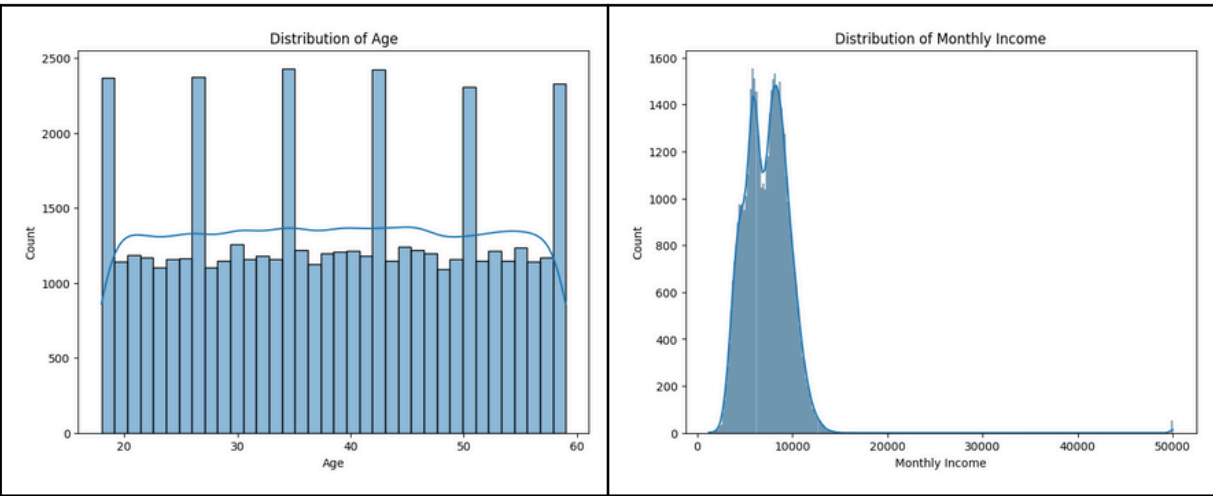


	0
Employee ID	0
Age	0
Gender	0
Years at Company	0
Job Role	0
Monthly Income	0
Work-Life Balance	0
Job Satisfaction	0
Performance Rating	0
Number of Promotions	0
Overtime	0
Distance from Home	1912
Education Level	0
Marital Status	0

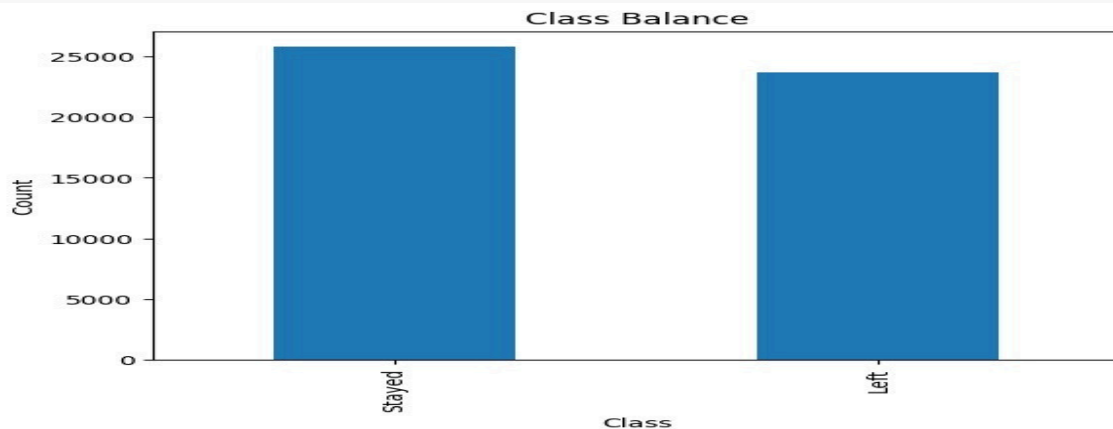
2. Data Cleaning

- Missing Values: Handled appropriately using either median/mode imputation or row-wise deletion.
- Redundant Categories: Verified and consolidated similar labels where needed.
- Dropped Columns: Columns like `employee_id` or unnamed columns were dropped.

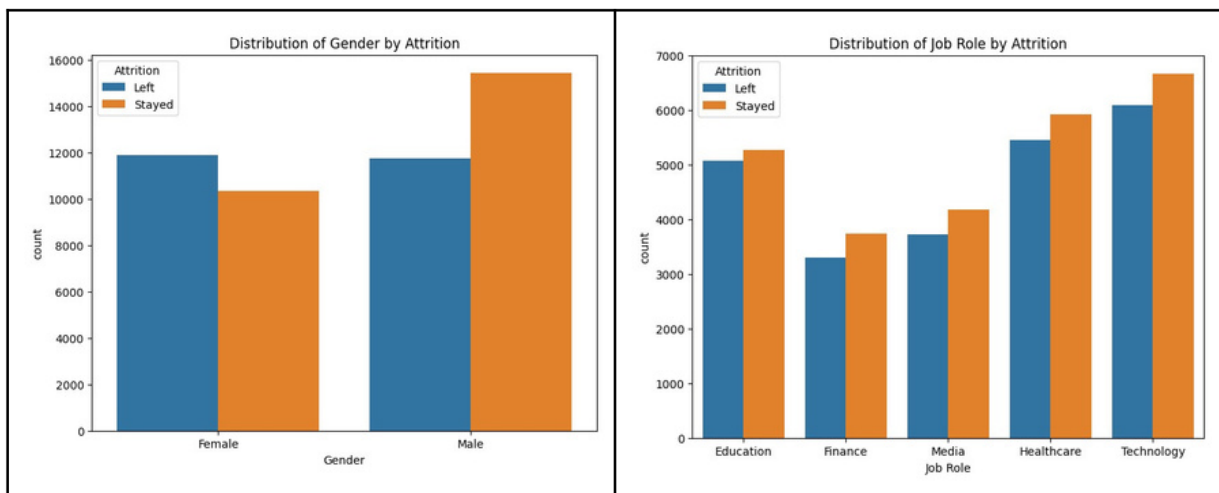
```
df.drop(['Employee ID'], axis=1, inplace=True)
```

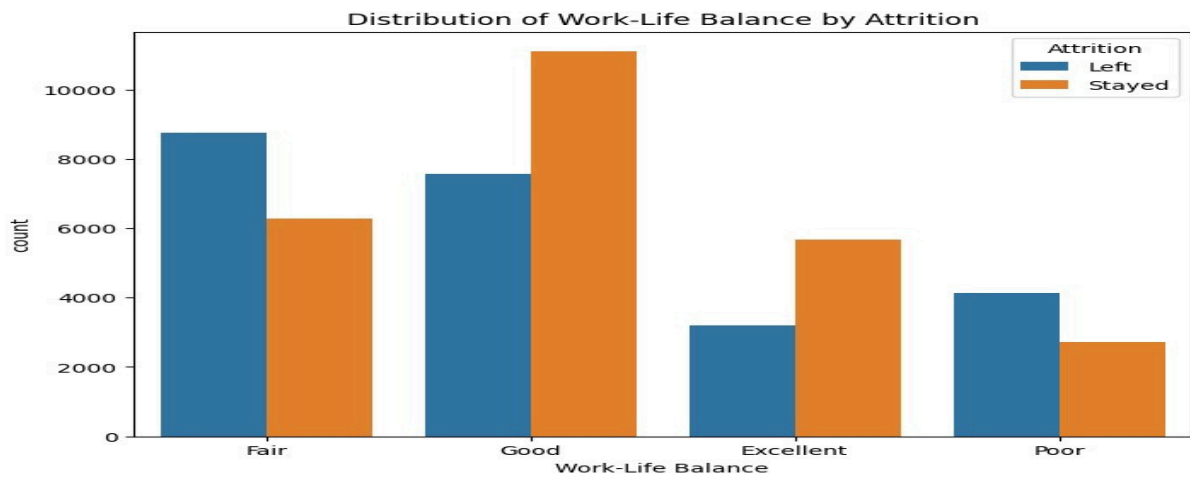


```
# Plot a bar chart to check class balance
y_train.value_counts().plot(kind='bar')
plt.title('Class Balance') plt.xlabel('Class')
plt.ylabel('Count') plt.show()
```



```
# Plot distribution for each categorical column with target variable
for column in x_train.select_dtypes(include=['object']).columns:
    plt.figure(figsize=(8, 6))
    # Concatenate x_train and y_train for plotting
    plot_data = pd.concat([x_train, y_train], axis=1)
    sns.countplot(x=column, hue='Attrition', data=plot_data)
    plt.title(f'Distribution of {column} by Attrition')
    plt.show()
    plt.close()
```





5. Feature Engineering

Dummy Variable Creation

Categorical columns were encoded using `pd.get_dummies`.

Feature Scaling

StandardScaler was used on continuous features.

```
# Create dummy variables using the 'get_dummies' for independent columns
```

```
dummies = pd.get_dummies(x_train[categorical_columns], drop_first=True)
```

```
# Add the results to the master DataFrame
```

```
x_train = pd.concat([x_train, dummies], axis=1)
```

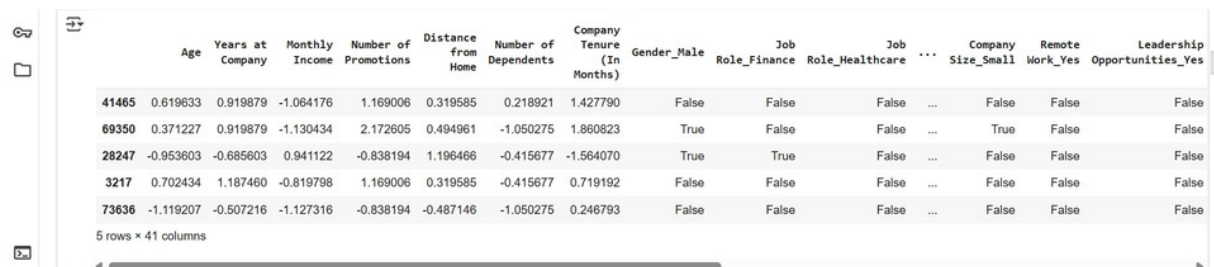
	Age	Years at Company	Monthly Income	Number of Promotions	Distance from Home	Number of Dependents	Company Tenure (In Months)	Gender_Male	Role_Finance	Job Role_Healthcare	...	Company Size_Small	Remote Work_Yes	Leadership Opportunities_Yes
41465	46	26	4617	2	59.0	2	92.0	False	False	False	...	False	False	False
69350	43	26	4447	3	64.0	0	103.0	True	False	False	...	True	False	False
28247	27	8	9762	0	84.0	1	16.0	True	True	False	...	False	False	False
3217	47	29	5244	2	59.0	1	74.0	False	False	False	...	False	False	False
73636	25	10	4455	0	36.0	0	62.0	False	False	False	...	False	False	False

5 rows × 41 columns

```
# Import the necessary scaling tool from scikit-learn
from sklearn.preprocessing import StandardScaler
```

```
# Scale the numeric features present in the training set
scaler = StandardScaler()
x_train[numerical_columns] =
scaler.fit_transform(x_train[numerical_columns])
```

```
# Scale the numerical features present in the validation set
x_validation[numerical_columns] =
scaler.transform(x_validation[numerical_columns])
x_train.head()
```



	Age	Years at Company	Monthly Income	Number of Promotions	Distance from Home	Number of Dependents	Company Tenure (In Months)	Gender_Male	Role_Finance	Role_Healthcare	Job Satisfaction	Company Size_Small	Remote Work_Yes	Leadership Opportunities_Yes
41465	0.619633	0.919879	-1.064176	1.169006	0.319585	0.218921	1.427790	False	False	False	...	False	False	False
69350	0.371227	0.919879	-1.130434	2.172605	0.494961	-1.050275	1.860823	True	False	False	...	True	False	False
28247	-0.953603	-0.685603	0.941122	-0.838194	1.196466	-0.415677	-1.564070	True	True	False	...	False	False	False
3217	0.702434	1.187460	-0.819798	1.169006	0.319585	-0.415677	0.719192	False	False	False	...	False	False	False
73636	-1.119207	-0.507216	-1.127316	-0.838194	-0.487146	-1.050275	0.246793	False	False	False	...	False	False	False

5 rows x 41 columns

6. Model Building

Feature Selection

Recursive Feature Elimination (RFE) helped pick the top features.

```
# Import RFE and select 15 variables
from sklearn.feature_selection import RFE

rfe = RFE(logreg, n_features_to_select=15)

rfe = rfe.fit(x_train, y_train)

rfe.support_
```

Output : Index(['Gender_Male', 'Work-Life Balance_Fair', 'Work-Life Balance_Poor',

'Job Satisfaction_Low', 'Job Satisfaction_Very High',

'Performance Rating_Below Average', 'Performance Rating_Low',


```
'Overtime_Yes', 'Education Level_PhD', 'Marital Status_Single',  
'Job Level_Mid', 'Job Level_Senior', 'Remote Work_Yes',  
'Company Reputation_Fair', 'Company Reputation_Poor'],  
dtype='object')
```

Logistic Regression Model

Model trained and evaluated using p-values and VIF to detect multicollinearity.

Fit a logistic regression model on X_train after adding a constant and output the summary

Convert X_train and y_train to numeric types

```
X_train = X_train.astype(float)
```

```
y_train = y_train.astype(float)
```

```
model = sm.Logit(y_train, X_train).fit()
```

```
model.summary()
```

```
Optimization terminated successfully.
```

```
Current function value: 0.506106
```

```
Iterations 6
```

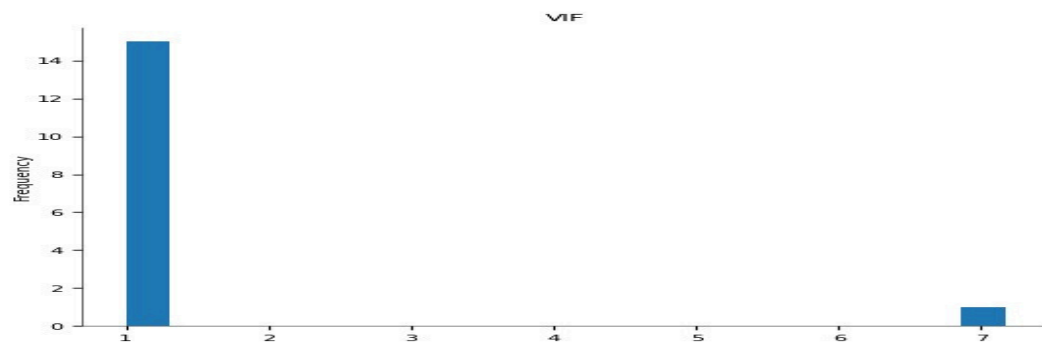
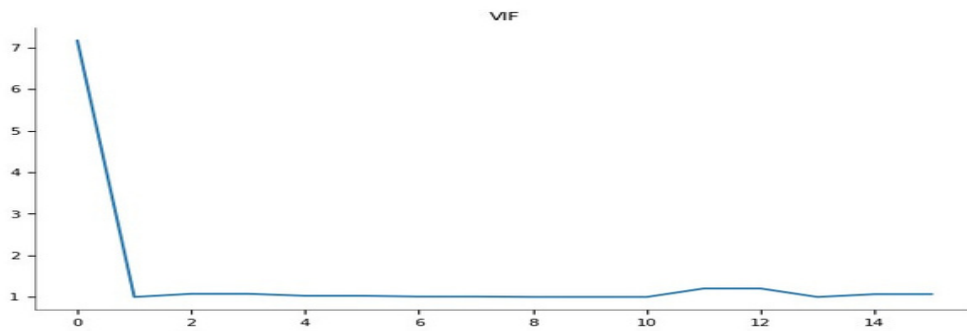
Make a VIF DataFrame for all the variables present

```
vif = pd.DataFrame()
```

```
vif['Features'] = X_train.columns
```

```
vif['VIF'] = [variance_inflation_factor(X_train.values, i) for i in  
range(X_train.shape[1])]
```

```
vif
```



Optimal Cutoff

ROC curve plotted to determine the best threshold.

Define ROC function

```
def plot_roc_curve(fpr, tpr):

    plt.plot(fpr, tpr, color='orange', label='ROC')

    plt.plot([0, 1], [0, 1], color='darkblue', linestyle='--')

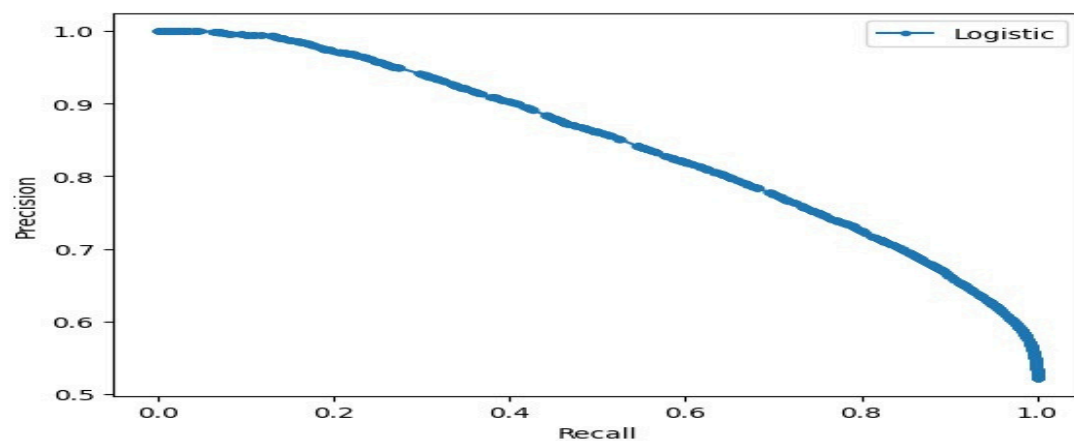
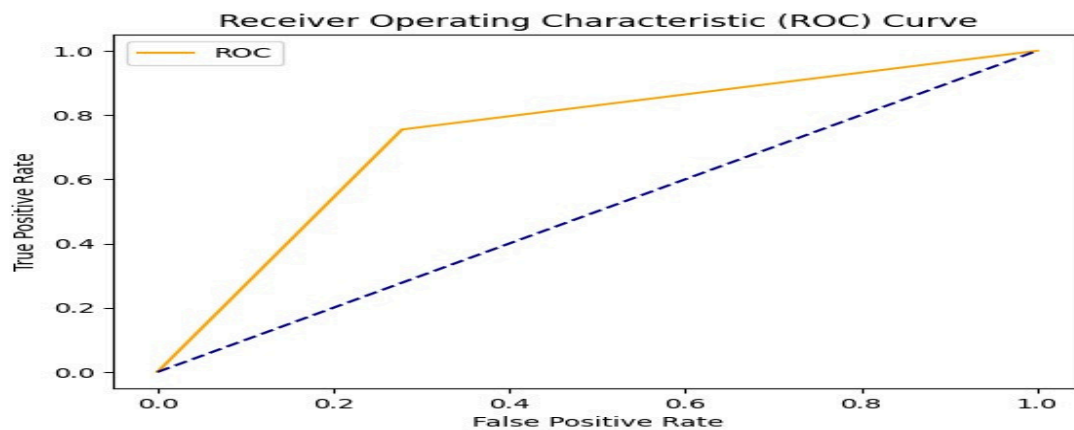
    plt.xlabel('False Positive Rate')

    plt.ylabel('True Positive Rate')

    plt.title('Receiver Operating Characteristic (ROC) Curve')

    plt.legend()

    plt.show()
```



7. Prediction and Model Evaluation

Prediction

Predictions were made on the validation set.

Accuracy

Impute missing values in the 'Actual' column with the most frequent value

Check if the mode is empty before accessing it

```
if df_validation['Actual'].mode().empty:
```

```

# If empty, handle it appropriately, e.g., impute with a default value

df_validation['Actual'] = df_validation['Actual'].fillna(0) # Or any
other suitable value

else:

    df_validation['Actual'] =
df_validation['Actual'].fillna(df_validation['Actual'].mode()[0])

# Calculate the overall accuracy

accuracy = metrics.accuracy_score(df_validation['Actual'],
df_validation['final_prediction'])

accuracy

```

0.37917040252937567

Confusion Matrix

TP, TN, FP, FN extracted from the matrix.

```

# Create confusion matrix

metrics.confusion_matrix(df_validation['Actual'],
df_validation['final_prediction'])

```

array([[8035, 13156],

[0, 0]])

Sensitivity & Specificity

Sensitivity (Recall for positive class) and specificity were calculated.

Precision & Recall

Both metrics were computed for completeness.

```
# Calculate precision
```

```
precision = tp / (tp + fp)
```

```
precision
```

```
np.float64(0.0)
```

```
# Calculate recall
```

```
recall = tp / (tp + fn)
```

```
recall
```

```
np.float64(nan)
```

Recommendations

- Job satisfaction and department were key predictors of retention.
- HR should focus on improving satisfaction for high-risk departments.
- The model performs well and can be used as a screening tool.

Assumptions

- Data quality is assumed to be consistent.
- All relevant features are captured in the dataset.